Final Project Report: Privacy-Preserving Sealed-Bid Auctions

Aarav Oswal (230012) Ch. Rishitha (230311)
R. Charan (230819) Poorvie Sadagopan (230759)
CS670 Project Group
Department of Computer Science and Engineering
Indian Institute of Technology Kanpur

November 2026

Abstract

We implement a sealed-bid auction, supporting both first-price and second-price (Vickrey) rules, in which no single party ever sees an individual bid. Bidders split their bids into secret shares held by three servers, which jointly evaluate the Kurosawa–Ogata bit-slice maximum circuit on MP-SPDZ using 3-party replicated secret sharing (semi-honest, honest majority). The compiled program opens exactly two values: the winner's index and the clearing price. We give an oblivious, branch-free formulation of the scan, an oblivious lowest-index tie-break that hides who tied, and a second oblivious scan for the Vickrey price. The circuit is implemented in both the arithmetic domain ($\mathbb{Z}_{2^{64}}$) and the binary domain (XOR-shared bits); a knockout tournament of secure comparisons in each domain serves as the baseline. Correctness is validated by 147 automated tests against a plaintext reference. Across 152 benchmark configurations, our cost model predicts the compiler's AND count exactly; moving to the binary domain reduces communication by 61×; and the tournament baseline, whose round count grows only logarithmically in the bid length, is faster on every realistic network. We discuss what this means for the round-complexity argument in our literature survey, and report the overhead of malicious security.

1 Introduction

A classical sealed-bid auction needs a trusted auctioneer who opens every envelope. That auctioneer learns every bidder's valuation, and in a Vickrey auction can inflate the clearing price undetectably. Our literature survey traced how cryptography removes this trust: first by splitting the auctioneer across two non-colluding servers (Naor, Pinkas and Sumner [10]), then across several servers holding secret shares (Bogetoft et al. [3, 4]), whose system ran the Danish sugar-beet auction of 2008. The mid-term report committed us to implementing this secret-sharing approach, with the bit-slice circuit of Kurosawa and Ogata [8] as its engine.

This report describes the finished system. Its privacy target, stated in the survey and kept unchanged, is:

> *The protocol reveals the identity of the winner and the clearing price, and nothing else about any bid, under a semi-honest adversary corrupting at most one of the three servers.*

Contributions.

- A working 3-server implementation on MP-SPDZ [7] of first-price and Vickrey auctions using the bit-slice circuit, with an oblivious tie-break and a reveal audit showing that exactly the winner and the price are opened (section 3).

- An exact cost model, $k(3n-1)$ ANDs and $k(\lceil \log_2 n \rceil + 2)$ rounds per scan, matched exactly by the compiler across all configurations (section 6).
- The same circuit in the binary domain, which cuts communication by 61×, plus a comparison-tournament baseline in both domains for a fair evaluation.
- An honest comparison: on modern MPC the tournament baseline needs far fewer rounds, which qualifies the survey's claim that the bit-slice scan wins on round complexity (section 7).
- A tested, reproducible code base (Linux and Windows via WSL) with 147 automated tests.

2 Background

Replicated secret sharing. We use the 3-party protocol of Araki et al. [1]. A value x is split into three random shares with $x = x_0 + x_1 + x_2$, and server p holds the pair (x_p, x_{p+1}) (fig. 1). Each pair alone is uniformly random, so one server learns nothing. Additions and multiplications by public constants are local. A multiplication costs each server one message of one ring element to its neighbour, in one round. The scheme works over the ring $\mathbb{Z}_{2^{64}}$ ("arithmetic domain") or over single bits with XOR as addition and AND as multiplication ("binary domain"). It tolerates one semi-honest corruption; two colluding servers can reconstruct everything.

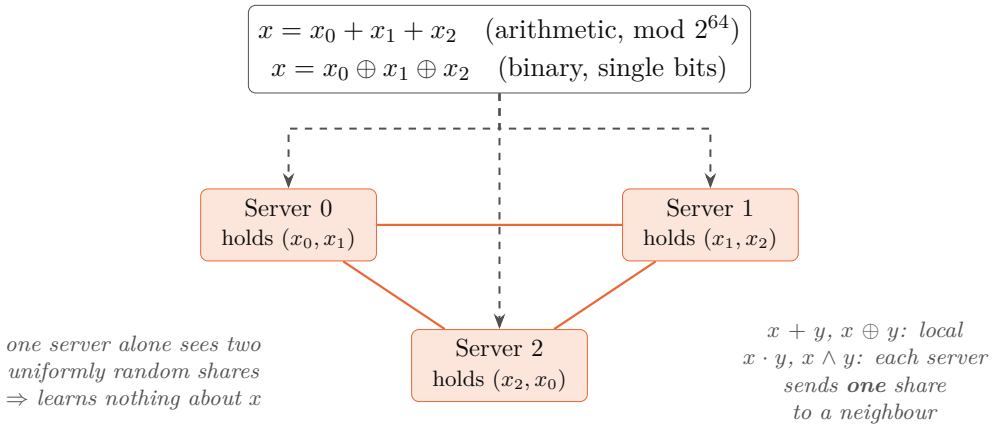


Figure 1: 3-party replicated secret sharing. Each server holds two of the three shares; one server alone sees only random values. Local operations are free; each multiplication (AND) costs one share sent per server.

MP-SPDZ. MP-SPDZ [7] compiles a high-level Python-like program (`.mpc`) into bytecode that any of its protocols can execute. It reports the exact number of multiplications ("triples"), rounds and opened values the program needs, which we use both to audit the output boundary and to validate our cost model. It also implements malicious variants of the same protocols (security with abort) [9, 5], which we use in section 6.5.

The bit-slice maximum. Kurosawa and Ogata [8] find the maximum of n hidden k-bit numbers by one scan from the most significant bit down, keeping a mask S of bidders who could still hold the maximum. At each bit, if some survivor has a 1, all survivors with a 0 are eliminated. Only AND and OR on bits are needed, so the scan can run in any MPC that supports them.

3 System Design

3.1 Architecture and threat model

Figure 2 shows the system. Each bidder secret-shares its bid and sends share p to server p. The three servers run the auction circuit under MP-SPDZ and open only the winner and the price.

The threat model is that of the survey (§7): servers and bidders follow the protocol (semi-honest) but may try to learn from what they see; at most one server is corrupted; collusion of two servers is outside the model.

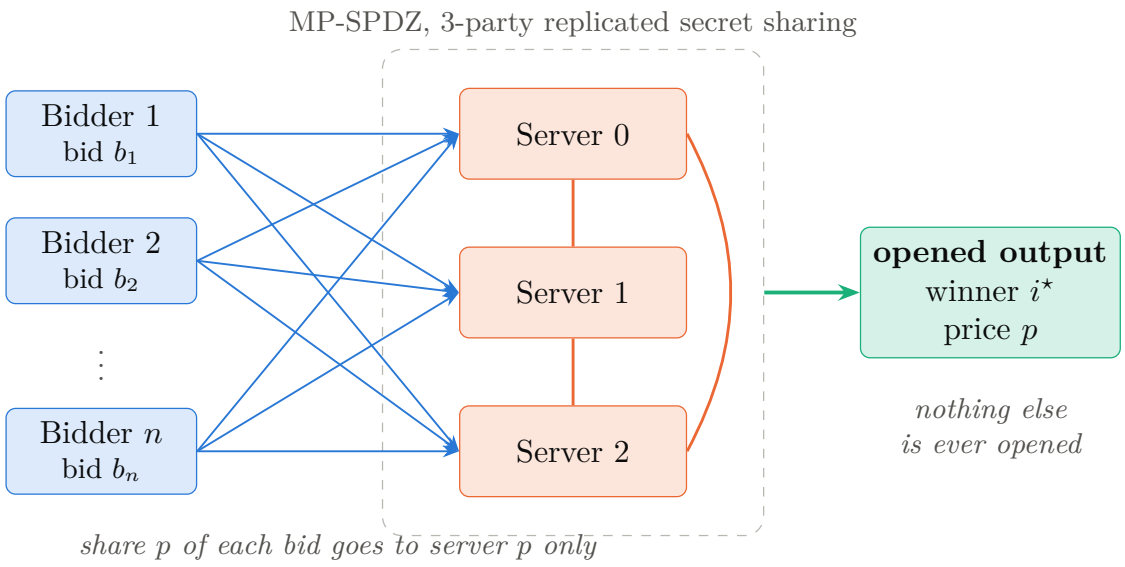


Figure 2: System architecture. Bidders are not servers: they only send random-looking shares. The servers compute jointly and open only the winner $i^\star$ and price p.

3.2 Input sharing

Bidders share their bids locally (`auction/sharing.py`). Each bidder first writes its bid in binary, which is trivial for the bidder, and then shares every bit separately:

- **arithmetic domain**: each bit $b_i[j]$ is split into three additive shares modulo 2^{64};
- **binary domain**: the bid is split into three k-bit XOR shares.

The servers therefore receive shares of the bits directly and never have to extract bits from a shared number. Each server feeds its shares to MP-SPDZ as private inputs, and the program adds (or XORs) the three inputs *inside* the computation, so the reconstructed bid exists only in shared form. Server p's input file holds a $k \times n$ matrix of shares in bit-major order (row j holds bit j of every bidder), so each bit position is read as one contiguous vector.

3.3 The oblivious bit-slice scan

The survey describes the update step as a branch on the check bit. A branch on secret data would leak it, so we write the step as pure arithmetic on shared 0/1 values (Algorithm 1, fig. 3). Let $t_i = S_i \cdot b_i[j]$ (survivor i has a 1 at bit j) and $c_j = \bigvee_i t_i$. The required update $S_i \leftarrow S_i \cdot (1 - c_j(1 - b_i[j]))$ expands to

$$S_i' \;=\; S_i - c_j\,(S_i - t_i), \tag{1}$$

which reuses t_i and needs one multiplication per bidder instead of two. The n-input OR is a balanced tree of $x \vee y = 1 - (1 - x)(1 - y)$, with $n - 1$ ANDs in $\lceil \log_2 n \rceil$ rounds (fig. 4). Since every c_j is a bit of the maximum, the maximum itself is $\sum_j c_j 2^j$ at no extra cost.

Figure 5 replays the survey's worked example ($A = 11$, $B = 13$, $C = 9$) through Algorithm 1. It also corrects Table 1 of the survey, which printed the mask after bits 1 and 0 as $\{\}$: when no survivor has a 1 the mask is carried forward, so it stays $\{B\}$. Our test suite checks this trace exactly (`test_worked_example_trace`).

```
Input: shared bits b_i[j] for bidders i = 0..n−1, bits j = 0..k−1
S ← (1, ..., 1)                                   // all bidders survive
for j = k − 1 downto 0:
    t ← S · b[j]                                   // n ANDs, vectorised
    c_j ← 1 − ∏_i (1 − t_i)                        // OR tree:  n-1 ANDs
    S ← S − c_j (S − t)                            // n ANDs, Eq.  (1)
return mask S, maximum bits (c_{k−1}, ..., c_0)
```

Algorithm 1: Oblivious bit-slice scan (`bitslice_scan` in `mpc/auction.mpc`). No step depends on a secret value, so the compiled circuit is fixed by n, k and the mechanism.

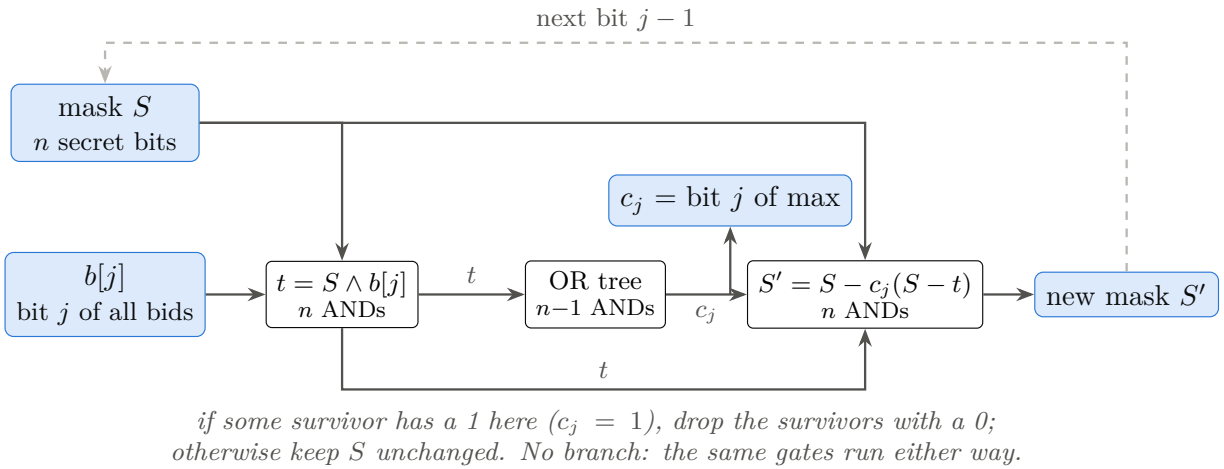


Figure 3: One step of the scan. All wires carry secret shares; the same gates run whatever the data, so the step is oblivious.

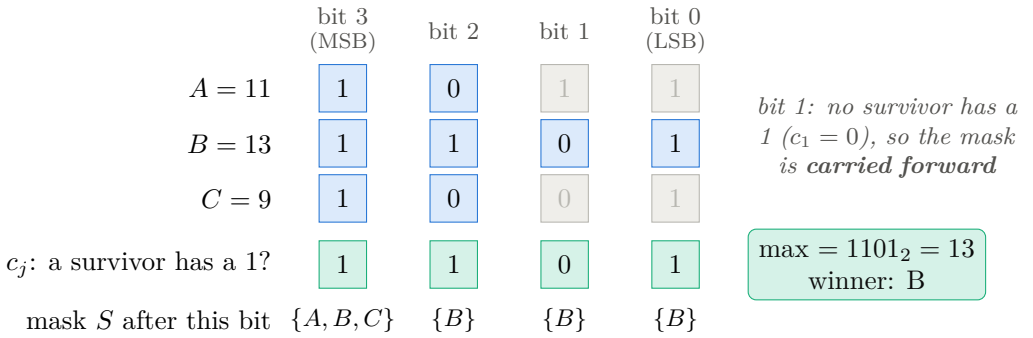


Figure 5: The scan on the survey's example. Blue cells: bidder still a survivor at that bit. This corrects the last two rows of Table 1 in the survey.

3.4 Ties: an oblivious lowest-index winner

When several bidders share the maximum, the final mask contains all of them. Opening it would reveal who tied, a leak the privacy target forbids. Instead we keep only the first survivor, $w_i = S_i \prod_{l<i}(1-S_l)$, using a Hillis–Steele prefix product in $\lceil \log_2 n \rceil$ rounds, and open only $\sum_i i \cdot w_i$ (fig. 6). The tournament baseline uses the same tie rule, so all programs produce identical outputs.

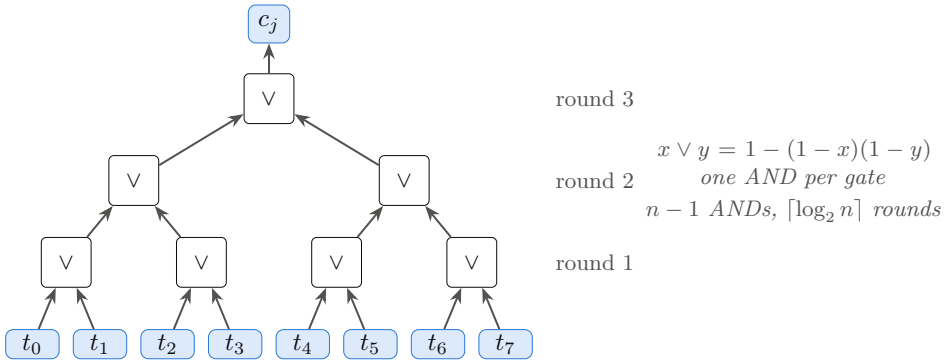


Figure 4: The n-input OR computed at every bit position ($n = 8$).

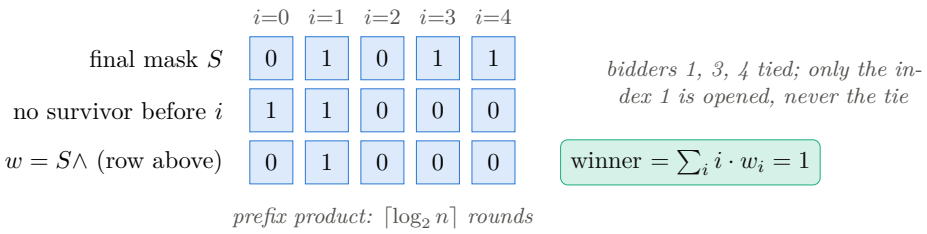


Figure 6: Oblivious lowest-index tie-break: three bidders tie, but only the index of the first is opened.

3.5 The Vickrey extension

For the second price we remove the winner obliviously and scan again (fig. 7): every bit of the winner is zeroed with $b_i[j] \leftarrow b_i[j]\,(1 - w_i)$, and the new maximum is the second-highest bid. The first-scan bits c_j, which encode the winning bid, are never opened. Two edge cases come out right without special handling: tied top bids give a price equal to the top bid, and a single bidder pays 0. The cost roughly doubles.

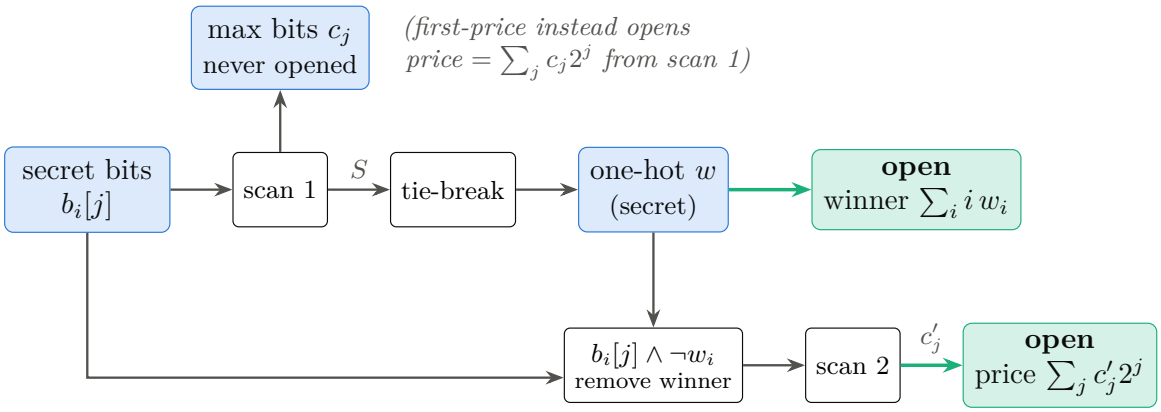


Figure 7: Vickrey pipeline. Secret values in blue, opened values in green. Only the winner index and the second-scan maximum are opened.

3.6 The binary-domain instantiation

The scan only ever combines single bits, yet in $\mathbb{Z}_{2^{64}}$ each AND is a full 64-bit ring multiplication. `mpc/auction_bin.mpc` runs the identical circuit on XOR-shared bits under MP-SPDZ's `replicated-bin` protocol (fig. 8). Bit j of all n bidders is packed into one n-bit word, so each line of Algorithm 1 is a single word-wide operation, and eq. (1) becomes $S' = S \oplus (c_j \wedge (S \oplus t))$. XOR and NOT are free and each AND costs one bit per server. The winner index needs no arithmetic: since w is one-hot, bit m of the index is the XOR of the w_i whose index has bit m set.

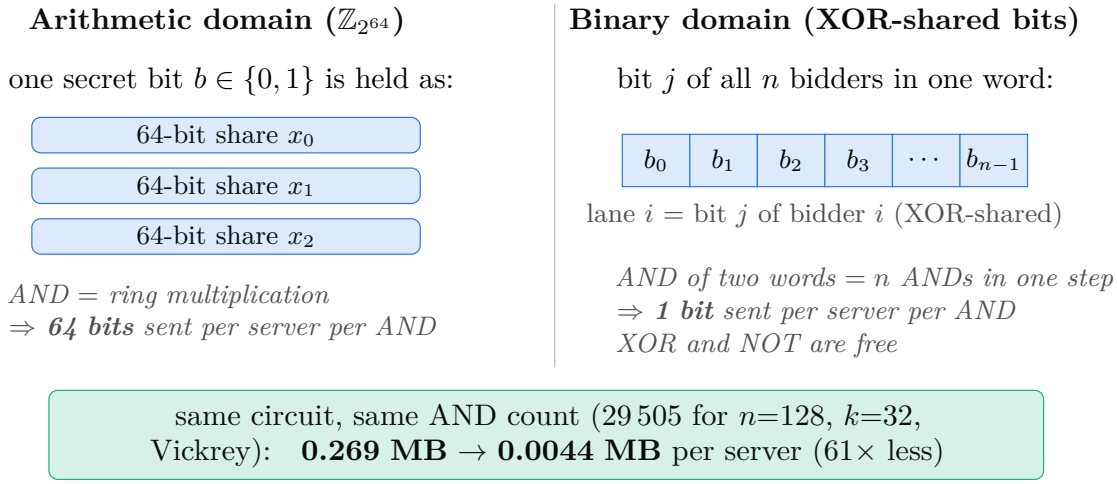


Figure 8: Arithmetic versus binary domain. The circuit and its AND count are identical; only the cost of each AND changes.

3.7 Baseline: a tournament of secure comparisons

To evaluate the bit-slice approach against the "naive" approach of survey §5.1, we implement a knockout tournament (fig. 9) using MP-SPDZ's built-in secure comparison, in both domains (`baseline_tournament.mpc`, `baseline_tournament_bin.mpc`). Each level compares neighbouring brackets in one vectorised step and multiplexes the running maximum, runner-up and index. Neighbours are paired as $(2i, 2i+1)$ and the right bracket wins only on a strict $>$, so the lowest index wins ties.

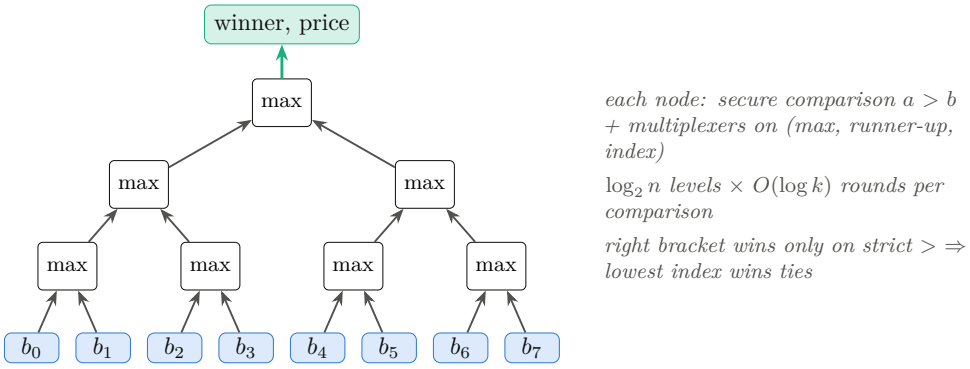


Figure 9: Comparison-tournament baseline ($n = 8$).

4 Security Analysis

Ideal functionality. $\mathcal{F}_{\text{auction}}$ receives every bid and returns $(i^\star, p)$, the lowest-index highest bidder and its price under the chosen rule. We argue that the protocol securely realises $\mathcal{F}_{\text{auction}}$ against a semi-honest adversary statically corrupting one server.

Simulation sketch. The corrupted server's view consists of three parts, each of which a simulator given only $(i^\star, p)$ can reproduce.

1. *Inputs.* For every bid (or bit), it holds one share s_p of $b = s_0 + s_1 + s_2$, where the bidder chose two shares uniformly at random. Any single share is therefore uniform on $\mathbb{Z}_{2^{64}}$ (or on k-bit strings for XOR sharing) and independent of b, so the simulator samples random values. Our test suite checks this empirically with a χ^2 test (`test_single_share_looks_uniform`).

2. *Computation.* The program is a fixed composition of local operations and secure multiplications of the replicated protocol, each simulatable for one corrupted party [1]. Control flow, number of rounds and message sizes depend only on the public parameters $(n, k,$ mechanism$)$, so nothing leaks through them.

3. *Outputs.* Exactly two values are opened. Given $(i^\star, p)$ the simulator chooses the honest parties' opening messages consistent with them.

By sequential composition, the corrupted server learns nothing beyond $(i^\star, p)$.

Reveal audit. `auction/audit.py` recompiles the programs and checks MP-SPDZ's count of opened values. The arithmetic program opens exactly 2 integers and the binary program reveals exactly two words, for every n and both mechanisms. The tournament baseline, by contrast, also opens masked values inside MP-SPDZ's comparison protocol.

Party	**What it sees**
Bidder	its own bid, its own shares, and the public output $(i^\star, p)$
One server	uniformly random shares of every bid, protocol messages that are themselves fresh random shares, and $(i^\star, p)$
Network observer	encrypted channels (MP-SPDZ uses TLS between servers); at most shares
Two colluding servers	*outside the model*: together they can reconstruct every bid

Table 1: Per-party view, matching the threat model of the survey (§7.2).

Inherent leakage. As stated in the survey, information implied by the output itself is not a leak: a first-price outcome reveals that every losing bid is at most p; a Vickrey outcome reveals that p is the second-highest bid. Ranks, orderings and ties among losers stay hidden.

Limitations of the guarantee. Bidders are trusted to submit well-formed inputs: a malicious bidder could share a "bit" equal to 2 and corrupt the result. Computing and opening $b_i[j](1-b_i[j])$, which is 0 for every valid bit, would detect this at the cost of nk extra multiplications. Under the malicious protocols of section 6.5, the *computation* is secure with abort against one malicious server, but a malicious server could still substitute its input share of a bid, since bidder shares are not authenticated. MP-SPDZ's client-input protocol, in which bidders mask their inputs with preprocessed random values, would close this gap.

5 Implementation

The code base is organised into the four components declared in the mid-term report (fig. 10); fig. 11 shows how they interact in one run.

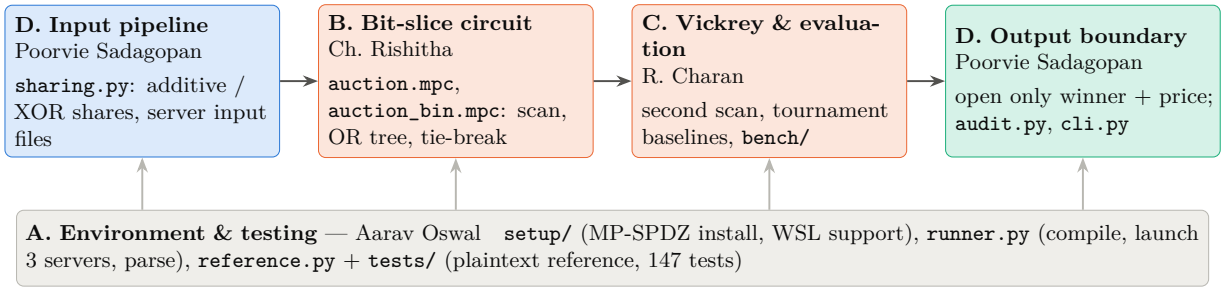


Figure 10: Code components and their owners, as declared in the mid-term report.

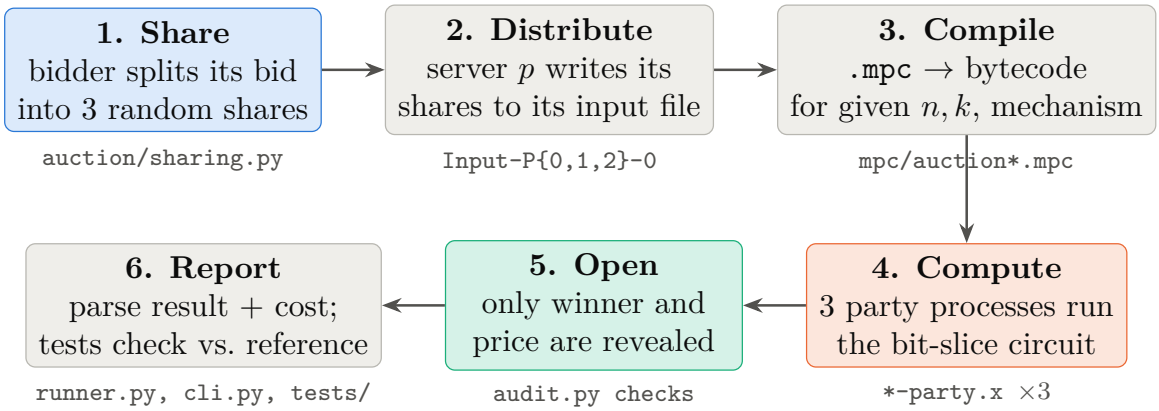


Figure 11: End-to-end workflow of one auction, with the file responsible for each step.

- **MPC programs** (`mpc/`): `auction.mpc` (arithmetic), `auction_bin.mpc` (binary), and the two tournament baselines. Each takes n, k and the mechanism as compile-time arguments. All operations are vectorised over the n bidders.
- **Runner** (`auction/runner.py`): compiles a program (cached by source timestamp), writes each server's input file to a private temporary directory, launches the three party processes on free localhost ports, and parses the result and MP-SPDZ's cost report. If one server crashes, it stops the others at once and retries on a fresh port.
- **References** (`auction/reference.py`): the plaintext auction rule, and a bit-level simulation of Algorithm 1 using the same arithmetic as the MPC.
- **Command-line demo** (`python -m auction`): runs any program and mechanism, can show what each server receives, and can check the result against the reference.
- **Setup** (`setup/setup.sh`): checks prerequisites, installs MP-SPDZ 0.4.3, creates the Python environment and runs a smoke test, on Linux or Windows via WSL2.

Testing. Figure 12 summarises the validation strategy. The plaintext rule is the ground truth. The bit-level simulation is checked exhaustively for $n \leq 3$, $k = 3$ and on 6 000 random cases. Every MPC program is run on nine edge cases (ties at the top, ties for second place, a single bidder, all-equal bids, all zeros, the maximum value, the winner last) and on random inputs with forced ties, for both mechanisms; the main programs are also run under malicious security. In total the suite has 147 tests and runs in about 9 seconds. The benchmark harness additionally checks every one of its 760 runs against the reference.

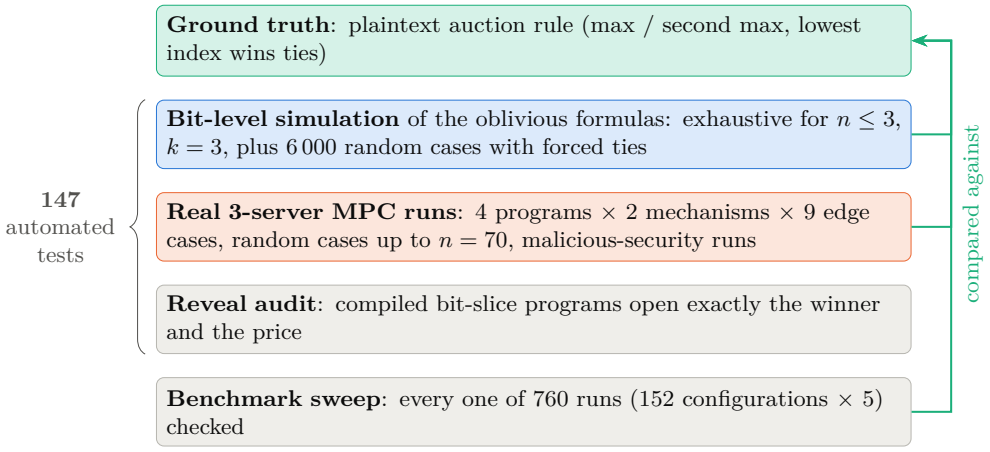


Figure 12: Validation strategy: every layer is checked against the plaintext auction rule.

rounds, Vickrey, $n = 32$	$k = 8$	$k = 32$	$k = 60$
Bit-slice, arithmetic	121	457	849
Bit-slice, binary	125	461	853
Tournament, arithmetic	50	62	62
Tournament, binary	66	86	86

Table 3: Rounds against the bid length. The bit-slice scan is linear in k; the comparison-based tournament is not.

6 Evaluation

Setup. All experiments run the three servers as separate processes on one machine (AMD Ryzen 7 6800H, 8 cores / 16 threads, 14 GB RAM, Ubuntu 24.04, MP-SPDZ 0.4.3). Bids are uniformly random; each configuration is repeated 5 times and we report the median local time. Rounds and data are MP-SPDZ's per-server figures (the protocols are symmetric) and include preprocessing. Because localhost has almost no latency, we also report a *modelled* time

$$T = T_{\text{local}} + \text{rounds} \times \text{RTT} + \text{bytes/bandwidth}$$

for a LAN (1 ms RTT, 1 Gbit/s) and a WAN (50 ms RTT, 100 Mbit/s). Rounds and data are measured; only the network is modelled. We compare four variants: bit-slice and tournament, each in both domains.

6.1 The cost model is exact

Counting operations in Algorithm 1 gives $k(3n - 1)$ ANDs and $k(\lceil \log_2 n \rceil + 2)$ rounds per scan. The tie-break adds $P(n) + n$ ANDs, where $P(n) = \sum_{d=1,2,4,\ldots<n}(n - d)$ is the cost of the prefix product. Vickrey adds the nk ANDs that zero the winner and a second scan. Table 2 compares these predictions with MP-SPDZ's compiler: they agree exactly, in both domains.

n	k	mechanism	predicted ANDs	compiled (arithmetic)	compiled (binary)
3	4	Vickrey	82	82	82
5	8	Vickrey	277	277	277
128	32	first	13 153	13 153	13 153
128	32	Vickrey	29 505	29 505	29 505
256	32	Vickrey	59 329	59 329	59 329

Table 2: Predicted versus compiled AND counts. Example: for $n = 128$, $k = 32$, first-price, $32 \cdot 383 + P(128) + 128 = 12\,256 + 769 + 128 = 13\,153$.

6.2 Scaling with the number of bidders and the bid length

Figure 13 shows rounds and data as n grows from 2 to 256 with $k = 32$. All variants grow linearly in data. The bit-slice rounds grow by about $k = 32$ per doubling of n, exactly the $k\lceil \log_2 n \rceil$ term, while the tournament adds only about 10 rounds per doubling. Table 3 isolates the bid length: the bit-slice scan is sequential in k, so its rounds grow linearly, whereas the tournament's rounds stop growing at all beyond $k = 32$ because MP-SPDZ's comparison needs only $O(\log k)$ rounds.

6.3 Arithmetic versus binary domain

Table 4 lists every variant at $n = 128$, $k = 32$. Moving the bit-slice circuit to the binary domain leaves its AND count and rounds unchanged but cuts its data from 0.269 MB to 0.0044 MB per server, a 61× reduction. The same move helps the tournament just as much (0.197 MB to



Figure 13: Rounds (top) and data per server (bottom, log scale) against the number of bidders, $k = 32$. Colour: algorithm; dashed lines with hollow markers: binary domain.

Variant (Vickrey)	ANDs	rand. bits	rounds	MB/server	local (ms)	LAN (s)	WAN (s)
Bit-slice, arithmetic	29 505	0	587	0.2688	19.4	0.61	29.4
Tournament, arithmetic	15 621	8 636	82	0.1972	5.1	0.09	4.1
Bit-slice, binary	29 505	0	591	0.0044	18.2	0.61	29.6
Tournament, binary	34 163	0	118	0.0049	4.3	0.12	5.9

Table 4: All variants at $n = 128$, $k = 32$, Vickrey. LAN and WAN times are modelled from the measured rounds and data. First-price costs about half (e.g. bit-slice arithmetic: 13 153 ANDs, 298 rounds, 0.138 MB).

0.0049 MB). Within the binary domain, bit-slice needs 14% fewer ANDs than the tournament (29 505 vs. 34 163) and about 10% less data; for small n the tournament is slightly lighter.

6.4 Modelled network time

On a real network, rounds dominate (fig. 14). With 50 ms of latency the bit-slice auction over 128 bidders takes about 29 s against 4–6 s for the tournament; even on a 1 ms LAN it takes 0.61 s against 0.09–0.12 s. The two bit-slice lines coincide: once data is this small, the domain no longer matters for time, only the rounds do.

6.5 Cost of malicious security

All four programs run unchanged under MP-SPDZ's malicious protocols (fig. 15). In the arithmetic domain, communication grows by a factor of about 6 (bit-slice at $n = 64$: 0.134 MB to 0.823 MB), with few extra rounds. In the binary domain, the malicious cost is almost constant at about 0.11–0.13 MB per server over the sizes we tested, because it is dominated by MP-SPDZ generating

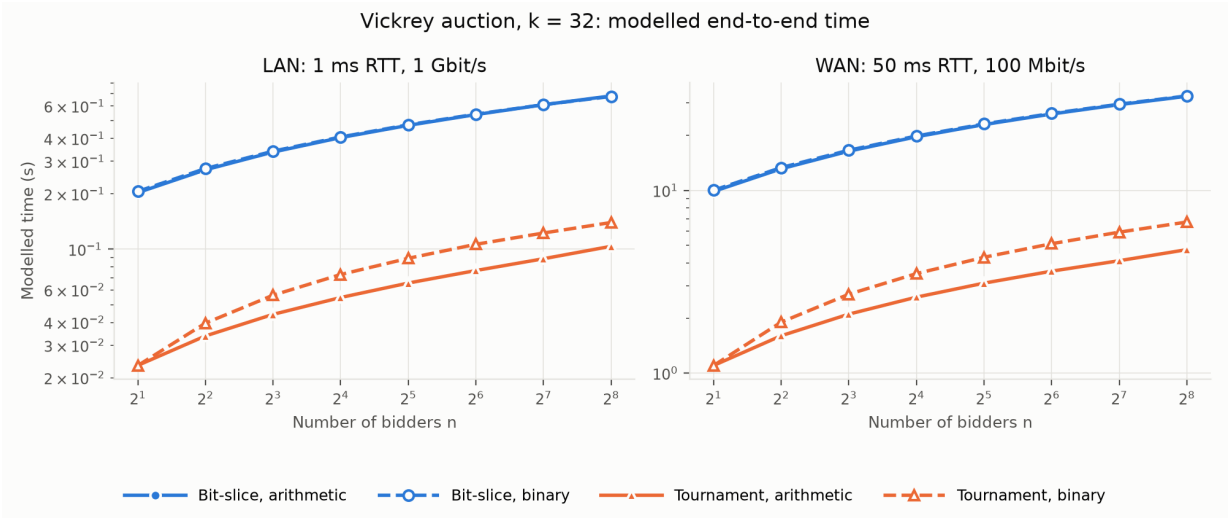


Figure 14: Modelled end-to-end time on a LAN and a WAN (Vickrey, $k = 32$, log scale).

and checking multiplication triples in large fixed batches, not by the circuit.

7 Discussion

Why the baseline wins. The survey (§5.4) argued that the bit-slice scan beats a tournament of comparisons on constants and round complexity, and Kurosawa and Ogata report a 7× speed-up over the best construction of their time. Our measurements do not support this on modern MPC. The scan has an inherent sequential dependency: bit $j-1$ cannot be processed before the mask from bit j is known, giving $\Theta(k \log n)$ rounds. MP-SPDZ's comparison uses preprocessed random bits and takes $O(\log k)$ rounds, so the tournament needs $O(\log n \cdot \log k)$. The claimed advantage was relative to comparison protocols of 2002, which needed far more rounds than today's.

Where bit-slice still has an edge. In the binary domain it needs the fewest ANDs and the least data for large n; it opens exactly two values, whereas the tournament opens $2n$ masked values inside its comparisons; it needs no comparison sub-protocol or preprocessed random bits, only multiplication; and the same scan carries over to very different trust models, such as the auctioneer-free SEAL protocol [2], where comparisons are not available. In a setting where bandwidth is the bottleneck and latency is negligible, binary bit-slice is the cheapest option we measured.

Domain matters more than algorithm. Choosing the binary domain changes communication by 61×; choosing between the two algorithms within one domain changes it by about 10%. For any boolean-heavy MPC workload, the domain is the first design decision to make.

Corrections to the literature survey.

- Table 1 printed the mask after bits 1 and 0 as {}; it should be $\{B\}$ (fig. 5).
- §5.2 describes the update as a branch on the check bit. A secure implementation must be branch-free; eq. (1) gives the oblivious form.
- §5.4's round-complexity argument holds against 2002-era comparisons but not against modern ones, as discussed above.

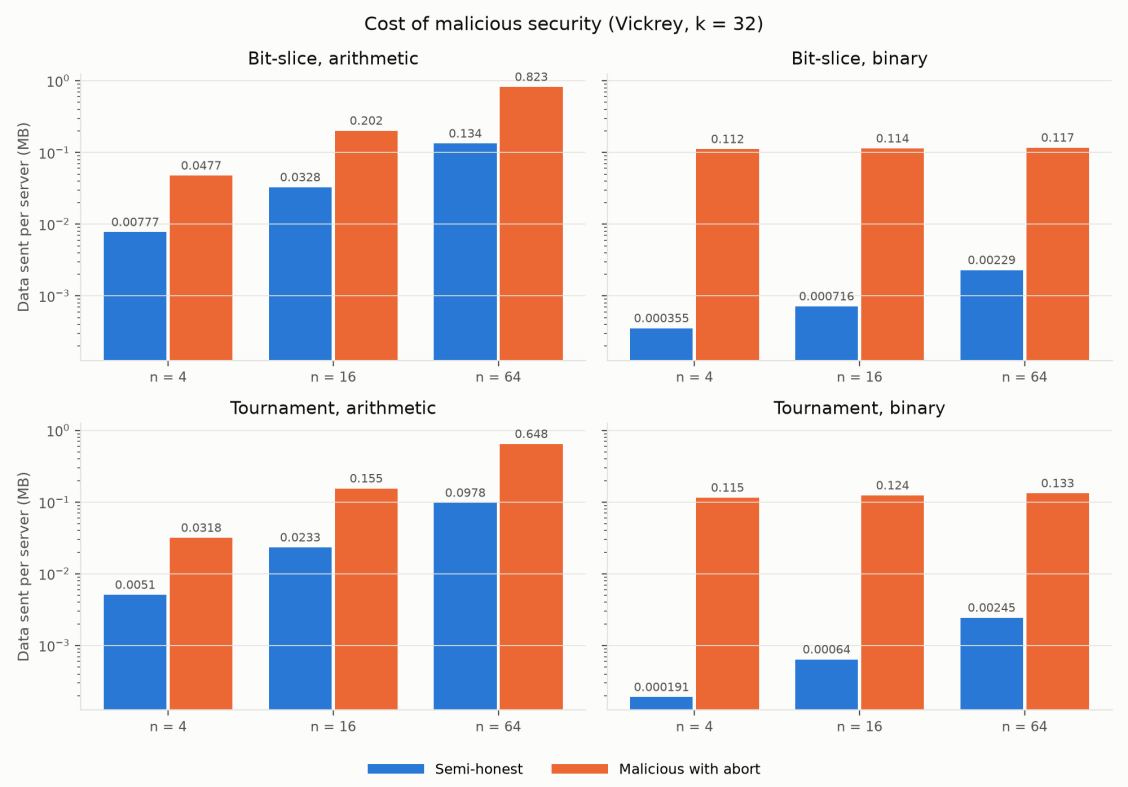


Figure 15: Data per server, semi-honest versus malicious security with abort (Vickrey, $k = 32$).

8 Limitations and Future Work

- **Malicious servers and input integrity.** The computation runs under malicious security, but bidder inputs should be authenticated, for example with MP-SPDZ's client interface, so that a server cannot substitute a share.
- **Public verifiability.** As surveyed in §6, commitments and zero-knowledge proofs (as in Galal and Youssef [6]) could let anyone verify the outcome; combining this with input privacy remains open in our system.
- **Fewer rounds for bit-slice.** Processing r bits per step (a radix-2^r scan) would divide the k term in the round count by r at the price of more ANDs per step.
- **More servers and real networks.** Shamir sharing with more servers would tolerate more corruptions; a deployment across real machines would replace our modelled network times with measured ones.
- **Auctioneer-free variant.** Implementing the scan with the SEAL protocol [2] would remove the servers altogether.

9 Individual Contributions

The work was divided into the four components declared in the mid-term report.

Aarav Oswal (230012): computation environment and testing
Installation and configuration of MP-SPDZ for the 3-server setting, including Windows support through WSL2 (`setup/`); the runner that compiles programs, launches the servers and parses their output (`runner.py`); the plaintext reference and bit-level simulation (`reference.py`); and the 147-test suite that validates every component against the reference.

Ch. Rishitha (230311): bit-slice maximum circuit
The oblivious bit-slice scan in both domains (`auction.mpc`, `auction_bin.mpc`): the branch-free mask update, the log-depth OR tree, the oblivious lowest-index tie-break, and the extraction of the maximum from the per-bit OR results.

R. Charan (230819): Vickrey extension and performance evaluation
The oblivious second scan for the Vickrey price; the comparison-tournament baselines in both domains; the benchmark harness and figures (`bench/`); and the analysis of scaling, network cost and malicious-security overhead.

Poorvie Sadagopan (230759): input and output pipeline
Bidder-side additive and XOR secret sharing and the server input layout (`sharing.py`); the output boundary that opens only the winner and price, verified by the reveal audit (`audit.py`); and the command-line demo (`cli.py`).

10 Conclusion

We built a privacy-preserving sealed-bid auction in which three servers compute the winner and the first- or second-price outcome without any of them seeing a bid, opening only the winner and the price. The bit-slice circuit is simple, oblivious and exactly predictable in cost, and moving it to the binary domain cuts its communication by 61×. Our evaluation also shows that its sequential dependency on the bid length makes it slower than a modern comparison-based tournament on any network with real latency, an honest correction to our own survey and a useful lesson about how far a 2002 efficiency claim carries into today's MPC.

Reproducing the results. `bash setup/setup.sh` installs everything; `.venv/bin/pytest -q` runs the tests; `.venv/bin/python bench/run_bench.py` and `bench/plot.py` regenerate every number and figure in section 6. Source code: `https://github.com/rcharan05/CS670_Project`.

References

[1] T. Araki, J. Furukawa, Y. Lindell, A. Nof, K. Ohara. High-throughput semi-honest secure three-party computation with an honest majority. In *ACM CCS*, pages 805–817, 2016. doi:10.1145/2976749.2978331.

[2] S. Bag, F. Hao, S. F. Shahandashti, I. G. Ray. SEAL: Sealed-bid auction without auctioneers. *IEEE TIFS*, 15:2042–2052, 2020. doi:10.1109/TIFS.2019.2955793.

[3] P. Bogetoft, I. Damgård, T. Jakobsen, K. Nielsen, J. Pagter, T. Toft. A practical implementation of secure auctions based on multiparty integer computation. In *Financial Cryptography*, LNCS 4107, pages 142–147, 2006. doi:10.1007/11889663_10.

[4] P. Bogetoft, D. L. Christensen, I. Damgård, M. Geisler, T. Jakobsen, M. Krøigaard, J. D. Nielsen, J. B. Nielsen, K. Nielsen, J. Pagter, M. Schwartzbach, T. Toft. Secure multiparty computation goes live. In *Financial Cryptography*, LNCS 5628, pages 325–343, 2009. doi:10.1007/978-3-642-03549-4_20.

[5] J. Furukawa, Y. Lindell, A. Nof, O. Weinstein. High-throughput secure three-party computation for malicious adversaries and an honest majority. In *EUROCRYPT*, 2017.

[6] H. S. Galal, A. M. Youssef. Verifiable sealed-bid auction on the Ethereum blockchain. In *Financial Cryptography Workshops*, LNCS 10958, pages 265–278, 2019. doi:10.1007/978-3-662-58820-8_18.

[7] M. Keller. MP-SPDZ: A versatile framework for multi-party computation. In *ACM CCS*, pages 1575–1590, 2020. doi:10.1145/3372297.3417872.

[8] K. Kurosawa, W. Ogata. Bit-slice auction circuit. In *ESORICS*, LNCS 2502, pages 24–38, 2002. doi:10.1007/3-540-45853-0_2.

[9] Y. Lindell, A. Nof. A framework for constructing fast MPC over arithmetic circuits with malicious adversaries and an honest-majority. In *ACM CCS*, 2017.

[10] M. Naor, B. Pinkas, R. Sumner. Privacy preserving auctions and mechanism design. In *ACM EC*, pages 129–139, 1999. doi:10.1145/336992.337028.